

**САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ**  
*Факультет прикладной математики – процессов управления*  
*Кафедра информационных систем*

**Маслакова Анастасия Андреевна**

**Выпускная квалификационная работа**  
**«Веб-платформа интерактивной визуализации и**  
**анализа многомерных численных алгоритмов»**

Уровень образования: бакалавриат

Направление: 01.03.02 Прикладная математика и информатика

Основная образовательная программа СВ.5005.2022: «Прикладная математика, фундаментальная информатика и программирование»

Научный руководитель  
доктор физ.-мат. наук, профессор  
Матросов Александр Васильевич

Рецензент  
Доцент кафедры моделирования экономических систем,  
кандидат физ.-мат. наук  
Ковшов Александр Михайлович

Санкт-Петербург  
2026

# Содержание

|                                                                                                  |    |
|--------------------------------------------------------------------------------------------------|----|
| Введение . . . . .                                                                               | 3  |
| Постановка задачи . . . . .                                                                      | 4  |
| Обзор литературы . . . . .                                                                       | 5  |
| Глава 1. Технология WebAssembly и высокопроизводительные вычисления в браузерной среде . . . . . | 7  |
| 1.1. Определение и назначение WebAssembly . . . . .                                              | 7  |
| 1.2. Архитектура и модель исполнения . . . . .                                                   | 7  |
| 1.3. Многопоточность в браузерной среде . . . . .                                                | 8  |
| 1.4. Методы компиляции нативного кода в WebAssembly . . . . .                                    | 9  |
| 1.5. Анализ производительности WebAssembly . . . . .                                             | 10 |
| 1.6. Спецификации BLAS и LAPACK . . . . .                                                        | 11 |
| Глава 2. Проектирование архитектуры платформы . . . . .                                          | 13 |
| 2.1. Требования к платформе и функциональные возможности . . . . .                               | 13 |
| 2.2. Выбор технологического стека . . . . .                                                      | 14 |
| 2.3. Архитектура прототипа . . . . .                                                             | 15 |
| 2.4. Структуры данных . . . . .                                                                  | 18 |
| 2.5. Интерфейсы взаимодействия . . . . .                                                         | 20 |
| Глава 3. Реализация прототипа . . . . .                                                          | 24 |
| 3.1. Подготовка вычислительного ядра . . . . .                                                   | 24 |
| 3.2. Подключение ядра к интерфейсному уровню . . . . .                                           | 25 |
| 3.3. Модули-солверы . . . . .                                                                    | 26 |
| 3.4. Режим рациональных дробей . . . . .                                                         | 28 |
| 3.5. Слой пошаговой визуализации и ввода . . . . .                                               | 29 |
| 3.6. Страницы-контейнеры . . . . .                                                               | 30 |
| 3.7. Модуль итоговых диаграмм . . . . .                                                          | 31 |
| Глава 4. Описание эксперимента и методика оценки платформы . . . . .                             | 33 |
| 4.1. Межбраузерный эксперимент . . . . .                                                         | 33 |
| 4.2. Метрики сравнения с внешними платформами . . . . .                                          | 35 |
| Глава 5. Результаты эксперимента и сравнительной оценки платформы . . . . .                      | 38 |
| 5.1. Результаты межбраузерного эксперимента . . . . .                                            | 38 |
| 5.2. Результаты сопоставления с внешними платформами . . . . .                                   | 40 |
| Выводы . . . . .                                                                                 | 43 |
| Заключение . . . . .                                                                             | 44 |
| Список литературы . . . . .                                                                      | 45 |

# Введение

Численные методы лежат в основе решений в широком круге прикладных областей. Их понимание необходимо как студентам технических направлений, осваивающим базовые алгоритмы, так и специалистам, решающим практические вычислительные задачи. Однако между этой потребностью и доступными инструментами для освоения численных методов имеется существенный разрыв.

MATLAB, Maple, Mathematica и другие традиционные среды численного анализа имеют высокий порог входа: их использование требует освоения собственного языка программирования, а коммерческие лицензии и сложность локальной настройки создают дополнительные ограничения. Будучи ориентированными на получение конечного результата, они не предоставляют встроенных универсальных средств пошаговой интерактивной визуализации вычислительного процесса, что затрудняет их использование для первичного освоения алгоритмов.

Существующие веб-ориентированные альтернативы решают проблему лишь частично. Вычислительные блокноты (Jupyter, Google Colab) предполагают работу через программный код и в большинстве случаев требуют серверной инфраструктуры выполнения. Онлайн-калькуляторы сфокусированы на отдельных операциях и не покрывают полный сценарий решения задачи. Большинство образовательных ресурсов ограничивается статическими иллюстрациями, исключая прямое взаимодействие пользователя с алгоритмом.

Решением этого разрыва выступает перенос существующих численных библиотек в браузерную среду с помощью технологии WebAssembly. В настоящей работе на базе этого подхода и современных средств визуализации проектируется и разрабатывается интерактивная веб-платформа. Реализованная архитектура совмещает высокую производительность расчётов над многомерными данными с инструментами пошаговой графической визуализации. Это обеспечивает прямое взаимодействие пользователя с вычислительным процессом и позволяет исследовать зависимость результатов от входных параметров в реальном времени.

# Постановка задачи

Целью данной работы является создание прототипа платформы для интерактивного анализа и визуализации многомерных численных алгоритмов, ориентированного на учебно-исследовательское применение.

Для достижения поставленной цели необходимо выполнить следующие задачи:

1. Провести анализ существующих инструментов и платформ для численного анализа и выработать на его основе требования к разрабатываемой платформе.
2. Исследовать технологию WebAssembly как основу для реализации высокопроизводительных вычислений в браузере.
3. Спроектировать архитектуру и реализовать прототип платформы с поддержкой интерактивной визуализации выбранных численных методов и возможностью работы с многомерными входными данными.
4. Разработать методику оценки платформы и провести экспериментальное исследование по двум контурам: внутреннему, охватывающему кроссбраузерное тестирование разработанного прототипа, и внешнему, охватывающему сравнительный анализ с существующими веб-платформами численных вычислений и визуализации.
5. Проанализировать полученные результаты и сформулировать рекомендации для дальнейшей доработки.

# Обзор литературы

Формирование современных подходов к созданию веб-приложений в области численного анализа опирается на работы в нескольких направлениях, включая анализ существующих платформ, технологию WebAssembly и её применение в задачах линейной алгебры, научной визуализации и образовательных средах. Анализ публикаций в этих направлениях позволяет обосновать выбор технологий и архитектурных решений, принятых в настоящей работе, а также определить место разрабатываемой платформы среди известных решений.

Состояние существующих инструментов численного анализа исследовалось в ряде академических работ. В исследовании [1] на основе опроса 156 респондентов и интервью с 20 специалистами выявлены девять групп системных проблем работы с блокнотами Jupyter, среди которых зависимость от серверной инфраструктуры, фрагментированность пользовательского сценария и ограниченная поддержка интерактивной визуализации. Систематический обзор [2] публикаций 2010-2020 годов о платформе GeoGebra показывает, что наибольший образовательный эффект даёт пошаговое визуальное представление, однако охват таких систем ограничен геометрией и элементарным анализом и не включает задачи многомерной линейной алгебры. На отдельных классах задач функциональность реализована в специализированных веб-инструментах [17, 18, 19, 20], однако каждый из них покрывает лишь часть сценариев и сохраняет указанные выше ограничения.

Технологические возможности, способные преодолеть выявленные ограничения, исследовались в ряде работ о WebAssembly. В статье [4] Rust-реализация матричных операций, скомпилированная в WASM через wasm-pack, превосходит JavaScript-аналог до 10 раз в Chrome и до 50 раз в Firefox на матрицах до  $1000 \times 1000$ , что объясняется более агрессивной JIT-оптимизацией V8 по сравнению со SpiderMonkey. В работе [5] на конфигурациях с матрицами до  $1024 \times 1024$  показано, что базовый WebAssembly без оптимизаций может уступать JavaScript (ускорение 0.41x), и только комбинация SIMD-векторизации и блочного доступа к памяти позволяет превзойти JavaScript до 1,64 раза.

Ряд публикаций подтверждает техническую жизнеспособность ком-

пиляции существующих C/C++ библиотек в WebAssembly с использованием Emscripten для реализации интерактивной научной визуализации непосредственно в браузере. Так, в работе [6] в WASM скомпилирован нативный C++-код алгоритма GPU-параллельного marching cubes для блочно-сжатых данных и продемонстрирована интерактивная визуализация изоповерхности набора данных объёмом около 1 ТБ в браузере с применением WebGPU, при этом поверхность из 137,5 миллиона треугольников вычисляется за 526 мс и рендерится со скоростью 30 FPS. В работе [7] описана кроссплатформенная визуализация научных симуляций через WebAssembly и WebGL без внешних зависимостей, где C-код компилируется через Emscripten в трёх режимах работы, включая нативный (OpenGL), гибридный (сервер и браузер) и полностью браузерный.

Опыт создания полноценных образовательных вычислительных сред в браузере представлен в статье [8]. Авторы реализовали браузерную среду разработки на Python, использующую Pyodide — порт CPython, скомпилированный в WebAssembly. Производительность системы составляет 3-5х от нативного CPython, что является приемлемым для образовательных задач. Практическое внедрение в школах Швейцарии подтверждает, что образовательные платформы на основе WebAssembly могут быть не только технически реализуемы, но и востребованы в учебном процессе.

Рассмотренные работы, с одной стороны, выявляют ограничения существующих платформ численного анализа, с другой стороны, демонстрируют возможности WebAssembly для их преодоления. Однако каждая из работ раскрывает только отдельный аспект и не затрагивает совокупности образовательного применения, интерактивной визуализации и охвата широкого класса численных задач. На этом основании актуальной становится задача объединения численных методов, интерактивной визуализации и образовательной функциональности в рамках одной программной системы, чему и посвящено настоящее исследование.

# **1   Технология WebAssembly и высокопроизводительные вычисления в браузерной среде**

## **1.1   Определение и назначение WebAssembly**

WebAssembly (Wasm) представляет собой открытый стандарт, определяющий переносимый бинарный формат инструкций для стековой виртуальной машины. Концептуальные требования к технологии сформулированы в основополагающей работе [3] и впоследствии формализованы в виде Core Specification W3C [21]. Wasm изначально проектировался как целевая платформа компиляции для языков с явным управлением памятью, таких как C, C++ и Rust, и распространяется в компактном бинарном виде, оптимизированном для быстрой загрузки и декодирования. Модуль компилируется в машинный код практически напрямую, что обеспечивает производительность, приближающуюся к нативной.

Появление WebAssembly связано с потребностью в ресурсоёмких вычислениях на стороне браузера, для которых JavaScript-движки V8, SpiderMonkey и JavaScriptCore даже при JIT-компиляции дают ограниченную предсказуемость и эффективность выполнения.

## **1.2   Архитектура и модель исполнения**

WebAssembly построен как виртуальная стековая машина со статической типизацией и обязательной валидацией модуля, исключающей нарушения типобезопасности во время исполнения. Спецификация [21] определяет четыре скалярных числовых типа: целые i32, i64 и числа с плавающей запятой f32, f64. Для задач линейной алгебры существенно, что тип f64 реализует стандарт IEEE 754 с детерминированной семантикой арифметики, за исключением битового представления NaN, что обеспечивает воспроизводимость численных результатов между хост-средами.

Линейная память представлена непрерывным расширяемым массивом байтов с плоской 32-разрядной адресацией, соответствующей целевому триплету `wasm32` для C/C++ и Rust, и изолирована от управляемой памяти JavaScript. Память организована постранично, размер страницы составляет 64 KiB, а расширение выполняется инструкцией `memory.grow`. Это задаёт гранулярность роста рабочего набора и теоретический потолок 4 GiB, который становится определяющим ограничением при размещении плотных матриц. Из JavaScript линейная память доступна как `ArrayBuffer` без копирования.

Модуль организован в виде секций, что допускает потоковую компиляцию параллельно с загрузкой бинарного файла. Исполнение происходит в изолированной среде, а взаимодействие с внешним миром обеспечивается только через явные импорты.

## 1.3 Многопоточность в браузерной среде

Традиционная однопоточная модель исполнения веб-браузеров, совмещающая обработку интерфейса и выполнение кода, накладывает ограничения на ресурсоёмкие вычисления: блокировка основного потока приводит к потере отзывчивости приложения. Использование механизма Web Workers позволяет выносить вычисления в изолированные фоновые потоки.

Для исключения накладных расходов на копирование больших массивов данных между потоками применяется объект `SharedArrayBuffer`, предоставляющий разделяемую область памяти. Синхронизация параллельного доступа к ней реализуется посредством атомарных операций объекта `Atomics`. В контексте WebAssembly инструментарий Emscripten реализует API POSIX `pthread`s поверх Web Workers, размещая линейную память Wasm-модуля в `SharedArrayBuffer`.

В современных браузерах доступ к разделяемой памяти жестко ограничен требованиями `cross-origin isolation`. Активация `SharedArrayBuffer` на уровне HTTP-протокола требует обязательной конфигурации заголовков `Cross-Origin-Embedder-Policy` (COEP) и `Cross-Origin-Opener-Policy` (COOP), формирующих изолированный контекст безопасности для защиты от несанкционированного доступа к данным.



## 1.4 Методы компиляции нативного кода в WebAssembly

WebAssembly как целевая платформа компиляции поддерживается широким набором инструментов, различающихся по исходным языкам, архитектуре компилятора и характеристикам генерируемого кода. Выбор конкретного стека определяет архитектуру бинарного модуля, объём сопутствующего рантайма и методику взаимодействия с хост-средой. Рассмотрим основные методы компиляции.

**АОТ-компиляция (ahead-of-time).** Трансляция исходного кода в бинарный модуль WebAssembly до запуска программы. Доминирующий способ доставки производительных Wasm-модулей в браузер и серверные рантаймы благодаря предсказуемой инициализации, компактному формату и глубоким оптимизациям на этапе сборки. Инструменты: Emscripten, LLVM/Clang, WASI SDK, Rust с `wasm-bindgen`, AssemblyScript.

**JIT-компиляция (just-in-time).** Компиляция во время исполнения с учётом контекста запуска и профиля нагрузки. В Wasm-экосистеме применяется не как способ получения модуля из исходного текста, а как механизм поздней оптимизации промежуточного представления или байткода внутри рантайма. Инструменты и движки: Wasmer JIT, Cranelift, JIT-пайплайны V8 и SpiderMonkey.

**Binary translation.** Перевод существующего бинарного кода, байткода или инструкций другой архитектуры в представление, исполняемое в WebAssembly. Применяется при недоступности исходников или экономической нецелесообразности переноса с уровня исходного кода. Примеры: CheerpX, QEMU/LLVM-ориентированные схемы динамической трансляции.

**Интерпретация.** В WebAssembly переносится не сама программа, а интерпретатор, исполняющий внешний бинарный код, байткод или сценарный язык инструкция за инструкцией. Используется для совместимости и быстрого переноса существующих экосистем без переработки их модели исполнения. Примеры: Pyodide (CPython), QuickJS, Lua-рантаймы.

**Управляемые рантаймы (Hybrid, VM → Wasm).** В Wasm переносится виртуальная машина целиком либо её компоненты в комбинации с АОТ-сборкой и поздней оптимизацией: часть кода интерпретируется, часть

компилируется заранее, критические участки оптимизируются рантаймом. Применяется для управляемых экосистем (Java, .NET) с сохранением байткода, системы типов и стека библиотек. Примеры: Blazor WebAssembly с рантаймом Mono, CheerJ.

Сравнительные характеристики методов приведены в таблице 1.1.

| Метод                | Вход                                  | Произв-сть     | Сложность      | Сильная сторона                             | Ограничение                                  |
|----------------------|---------------------------------------|----------------|----------------|---------------------------------------------|----------------------------------------------|
| АОТ                  | Исходный код                          | Высокая        | Средняя        | Глубокие оптимизации, компактный модуль     | Отдельный этап сборки                        |
| JIT                  | IR, байткод во время исполнения       | Средняя        | Высокая        | Оптимизация по реальному профилю выполнения | Замедляет холодный старт                     |
| Binary translation   | Готовый бинарь, машинный код, байткод | Средняя/низкая | Очень высокая  | Не требует доступа к исходникам             | Зависит от качества сопоставления архитектур |
| Интерпретация        | Бинарь, байткод, сценарный код        | Низкая         | Низкая/средняя | Быстрый перенос экосистем                   | Накладные расходы на каждой инструкции       |
| Управляемые рантаймы | Байткод VM, смешанный вход            | Средняя        | Высокая        | Совместимость со стеком библиотек платформы | Большой объём рантайма, медленный старт      |

Таблица 1.1: Сравнение методов компиляции в WebAssembly

## 1.5 Анализ производительности WebAssembly

Производительность WebAssembly зависит не только от самого формата, но и от характера программы, браузера, runtime-среды, компилятора и используемых оптимизаций. В работе [9], где сравнивались 41 C-бенчмарк, 9 пар WebAssembly/JavaScript-программ и 3 реальных приложения, показано, что WebAssembly не всегда автоматически превосходит JavaScript. Современные JIT-компиляторы эффективно оптимизируют JavaScript, тогда как прирост от JIT для WebAssembly в Chrome и Firefox оказался ограниченным. Кроме того, из-за модели линейной памяти WebAssembly в ряде случаев потреблял больше памяти, чем JavaScript [9].

В работе [10] дополнительно показано различие между микробенчмарками и реальными приложениями: на микробенчмарках WebAssembly в среднем был на 9.84% медленнее JavaScript и потреблял на 5.18% больше энергии, однако на реальных приложениях оказался на 17.24% быстрее и снизил энергопотребление на 24.34%. При сравнении с нативным кодом важным источником накладных расходов являются проверки границ линейной

памяти: в работе [11] на наборе SPEC CPU показано, что в браузере накладные расходы WebAssembly относительно native составляют в среднем 45% в Firefox и 55% в Chrome, с пиками до  $2.08\times$  и  $2.5\times$  на отдельных бенчмарках; результат существенно зависит от реализации runtime.

Таким образом, WebAssembly следует рассматривать не как универсальный способ автоматического ускорения программ, а как эффективный инструмент для переноса вычислительно нагруженного и заранее оптимизированного кода в веб-среду или изолированную runtime-среду. Наибольший выигрыш достигается в случаях, когда исходный код уже хорошо оптимизирован на уровне алгоритмов и работы с памятью, а компиляция выполняется с учётом особенностей WebAssembly. Для задач линейной алгебры это особенно важно, поскольку итоговая производительность определяется не только самим фактом использования WebAssembly, но и блочной организацией вычислений, локальностью обращений к памяти, качеством компиляции и поддержкой низкоуровневых оптимизаций.

## 1.6 Спецификации BLAS и LAPACK

BLAS (Basic Linear Algebra Subprograms) [24] представляет собой спецификацию интерфейсов для базовых операций линейной алгебры. Спецификация организована в три уровня:

- **Уровень 1** определяет операции вектор-вектор со сложностью  $O(n)$ : скалярное произведение, вычисление норм, масштабирование, линейные комбинации.
- **Уровень 2** определяет операции матрица-вектор со сложностью  $O(n^2)$ : умножение матрицы на вектор, решение треугольных систем.
- **Уровень 3** определяет операции матрица-матрица со сложностью  $O(n^3)$ : матричное умножение, ранговые обновления.

Операции третьего уровня обладают высоким соотношением вычислений к обращениям в память ( $O(n)$  операций на элемент данных), что позволяет эффективно использовать кэш-память и достигать производительности, близкой к пиковой.

LAPACK (Linear Algebra PACKage) [25] представляет собой библиотеку подпрограмм для решения задач линейной алгебры высокого уровня: факторизации матриц, решения систем линейных уравнений, вычисления собственных значений и сингулярного разложения. Подпрограммы LAPACK реализованы через вызовы BLAS, что позволяет автоматически наследовать оптимизации конкретной реализации BLAS для целевой платформы.

Соглашение об именовании подпрограмм LAPACK отражает тип данных и характер операции:

1. первая буква указывает на тип данных (например, d для double precision);
2. последующие буквы кодируют тип матрицы (ge для общей, sy для симметричной, po для положительно определённой);
3. окончание определяет выполняемую операцию (sv для решения системы, trf для факторизации).

OpenBLAS [26] является одной из наиболее эффективных и широко используемых реализаций стандартов BLAS и LAPACK с открытым исходным кодом. Библиотека содержит глубоко оптимизированные вычислительные ядра, написанные на языках C, Fortran и ассемблере, что обеспечивает высокую производительность на различных процессорных архитектурах (x86, ARM, RISC-V и других). Благодаря поддержке многопоточного исполнения и возможности адаптации под специфические ограничения целевых сред, OpenBLAS выступает в качестве промышленного стандарта для интеграции методов высокопроизводительной линейной алгебры в современные программные комплексы и кроссплатформенные системы.

## 2 Проектирование архитектуры платформы

### 2.1 Требования к платформе и функциональные возможности

При проектировании интерактивной веб-платформы основной задачей становится построение архитектуры, сочетающей в себе высокую производительность и модульность с наглядной визуализацией алгоритмов. В отличие от обычных вычислительных программ, такая система ориентирована не только на получение результата, но и на демонстрацию логики вычислительного процесса непосредственно в браузере, что определяет её образовательное назначение как инструмент изучения численных методов.

Исходя из этого назначения, к платформе предъявляются следующие функциональные требования:

- решение многомерных систем линейных алгебраических уравнений прямыми и итерационными методами;
- решение нелинейных систем уравнений итерационными методами;
- построение моделей аппроксимации и регрессии по набору наблюдений;
- гибкий ввод данных через матричные сетки, текстовые поля для уравнений и динамические таблицы наблюдений;
- пошаговая анимированная визуализация хода алгоритма с синхронной цветовой индикацией обрабатываемых элементов, управлением скоростью воспроизведения и выводом диагностических диаграмм.

К нефункциональным требованиям относятся: выполнение всех вычислений на стороне клиента, без серверного бэкенда; декомпозиция длительных алгоритмов на атомарные операции с асинхронными паузами,

обеспечивающая отзывчивость интерфейса при обработке матриц прикладной размерности; единый пользовательский сценарий взаимодействия для всех классов задач; возможность добавления нового численного метода без изменения существующих модулей.

Перечисленные требования определяют необходимость разделения платформы на несколько согласованных уровней. **Интерфейсный уровень** отвечает за ввод данных, навигацию и визуализацию результатов. **Прикладной уровень** управляет сценариями решения задач и координирует выполнение конкретных численных методов. **Вычислительный уровень** обеспечивает выполнение ресурсоёмких операций линейной алгебры. Подобная декомпозиция минимизирует связанность подсистем, обеспечивая высокую масштабируемость и упрощая интеграцию новых расчётных модулей.

## 2.2 Выбор технологического стека

Проектирование платформы интерактивной визуализации численных алгоритмов требует обоснованного выбора технологий, обеспечивающих выполнение ключевых требований: высокопроизводительные вычисления на стороне клиента, модульная архитектура интерфейса, быстрая начальная загрузка и возможность интерактивной визуализации промежуточных состояний алгоритмов. В данном разделе последовательно рассматриваются решения, принятые для каждого уровня технологического стека.

Основу вычислительного ядра платформы составляет технология WebAssembly (WASM), выбор которой аргументирован результатами анализа в Главе 1. В качестве базового инструментария компиляции выбран Emscripten SDK, что продиктовано возможностью прямой трансляции высокооптимизированных нативных библиотек линейной алгебры, в частности OpenBLAS и LAPACK, в бинарный формат WASM. Данный подход обеспечивает доступ к промышленно апробированным реализациям BLAS-операций и LAPACK-факторизаций без потери эффективности, достигая производительности, сопоставимой с нативными приложениями.

Для реализации пользовательского интерфейса и управления состоянием приложения выбран фреймворк React.js. Компонентно-ориентированный подход и механизм Virtual DOM обеспечивают синхро-

низацию визуального представления с результатами вычислений. Загрузка WASM-модуля выполняется асинхронно, а отзывчивость интерфейса при больших размерностях достигается за счёт декомпозиции алгоритма на атомарные операции с возвратом управления браузеру между шагами.

Визуализационный слой комбинирует две технологии: матричные сетки и пошаговые таблицы строятся как DOM-структуры с императивной подсветкой ячеек, тогда как диаграммы регрессии и аппроксимации отрисовываются средствами Canvas API без сторонних библиотек. Такое разделение позволяет повторно использовать стандартные средства браузера для табличного представления и резервировать растровый канвас для частой перерисовки графиков. Взаимодействие JavaScript-слоя с вычислительным ядром выполняется через линейную память WebAssembly, что минимизирует накладные расходы на передачу данных. На уровне инфраструктуры предусмотрены HTTP-заголовки COOP/COEP, формирующие изолированный контекст безопасности и оставляющие задел для перехода на pthread-сборку OpenBLAS в дальнейшем.

## 2.3 Архитектура прототипа

Настоящий раздел описывает архитектурные решения, принятые при проектировании: организацию взаимодействия между уровнями, механизм подключения алгоритмов, командный протокол вычислительного ядра и модель пошаговой визуализации.

Трёхуровневая архитектура платформы опирается на принципы MVP (Model–View–Presenter). Роль Model выполняет вычислительное ядро, инкапсулирующее вычисления и не зависящее от интерфейса и логики конкретного метода. Роль View принадлежит компонентам интерфейсного уровня, отвечающим за ввод данных и отображение результатов. Роль Presenter реализуют модули-солверы прикладного уровня, координирующие выполнение метода и управление визуализацией. Методы, основанные на стандартных процедурах линейной алгебры, включая прямые и итерационные методы решения линейных СЛАУ, а также методы аппроксимации и регрессии, делегируют вычисления ядру через операции BLAS/LAPACK. Нелинейные методы реализуются на прикладном уровне, возникающие при этом линейные под-

системы решаются его собственными средствами. Эквивалентные команды полного решения СЛАУ формируются лишь для отображения в журнале шагов в учебных целях. Компоненты интерфейса не взаимодействуют с ядром напрямую: страница-контейнер получает абстрактный интерфейс его вызова из инфраструктурного контекста и передаёт его солверу, тогда как формирование команд и интерпретация результатов выполняются на прикладном уровне.

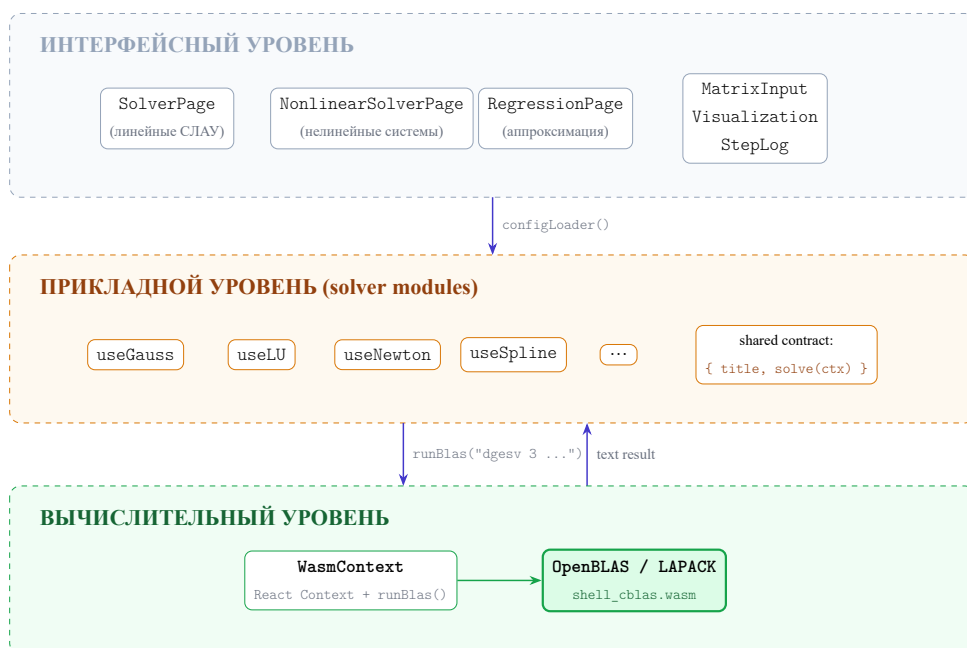


Рис. 2.1: Трёхуровневая архитектура платформы

Каждый численный метод оформлен как самодостаточный модуль общей структуры — контракт солвера. Контракт включает общие метаданные метода, набор полей, специфичных для класса задачи и страницы-контейнера, а также операцию решения, реализующую полный вычислительный сценарий. Операция решения получает контекст выполнения — структуру, агрегирующую входные данные задачи, инжектируемый интерфейс вызова ядра, императивные интерфейсы визуализации и журнала шагов, а для интерактивных страниц также разделяемый флаг пропуска анимации и флаг готовности ядра. Состав общих полей контракта приведён в таблице 2.1, а специфичные для класса задачи поля перечислены в таблице 2.2.



| Поле контракта                        | Обязат. | Назначение                                                                                                                                         |
|---------------------------------------|---------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Заголовок и описание метода           | да      | Отображаются в шапке страницы задачи                                                                                                               |
| Служебный префикс <code>prefix</code> | нет     | При наличии используется линейной страницей для генерации <code>id</code> полей ввода; при отсутствии берётся ключ маршрута                        |
| Базовая задержка между шагами журнала | нет     | Для линейных и нелинейных страниц задаётся полем <code>stepDelay</code> ; в <code>RegressionPage</code> используется фиксированное значение 400 мс |
| Дополнительные параметры метода       | нет     | $\epsilon$ , максимальное число итераций, релаксационный параметр и т.п.                                                                           |
| Операция решения                      | да      | Асинхронная процедура, получающая контекст выполнения                                                                                              |

Таблица 2.1: Общие поля контракта модуля-солвера

| Поле контракта                                    | Обязат. | Назначение                                                                                                                                                                 |
|---------------------------------------------------|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Линейные СЛАУ и нелинейные системы (общие)</i> |         |                                                                                                                                                                            |
| Размерности задачи и предзаполненного примера     | да      | Начальный размер $n$ матричной сетки и размер предзаполненного примера, отображаемого при загрузке страницы                                                                |
| <i>Линейные СЛАУ</i>                              |         |                                                                                                                                                                            |
| Предзаполненный пример                            | нет     | Матрица коэффициентов и вектор правой части, подставляемые в поля ввода при открытии страницы                                                                              |
| Подпись поля ввода матрицы                        | нет     | Заголовок над сеткой ввода; используется при дополнительных требованиях к матрице (например, симметричность и положительная определённость для метода Холецкого)           |
| <i>Нелинейные системы</i>                         |         |                                                                                                                                                                            |
| Предзаполненная система уравнений                 | нет     | Список выражений $F_i(x_1, \dots, x_n)$ , отображаемых в полях ввода уравнений при загрузке страницы                                                                       |
| Начальное приближение                             | нет     | Стартовый вектор $x^{(0)}$ итерационного процесса, подставляемый в поля начальной точки                                                                                    |
| <i>Аппроксимация и регрессия</i>                  |         |                                                                                                                                                                            |
| Число входных признаков                           | нет     | Начальное количество столбцов-признаков $x_j$ в таблице наблюдений; для одномерных методов равно единице                                                                   |
| Предзаполненный набор наблюдений                  | нет     | Список строк $(\mathbf{x}, y)$ , подставляемых в таблицу наблюдений при открытии страницы                                                                                  |
| Минимально допустимое число точек                 | нет     | Нижняя граница числа наблюдений, при которой допустим запуск метода; задаётся как функция числа признаков (например, $n + 1$ для линейной регрессии, $\geq 3$ для сплайна) |

Таблица 2.2: Поля контракта, специфичные для класса задачи

В отличие от подхода, при котором вычислительное ядро выполняет метод целиком и возвращает финальный результат, в данной архитектуре алгоритм реализован как последовательность элементарных операций на прикладном уровне. Каждая операция может включать один или несколько вызовов ядра, обновление визуального состояния и запись в журнал. Между операциями вставляются асинхронные паузы, которые передают управление

браузеру и позволяют отрисовать промежуточное состояние до продолжения вычислений.

Прямые методы (Гаусса, LU, QR) автоматически выбирают режим вычислений по типу входных данных. При целочисленных коэффициентах шаги выполняются на прикладном уровне в режиме рациональных дробей, промежуточные преобразования отображаются обыкновенными дробями без округлений, что допускает ручную проверку каждого шага. Числитель и знаменатель хранятся в типе `Number`, поэтому при росте коэффициентов произвольная точность не гарантируется. В вещественном режиме выбор ведущего элемента, элиминация и обратная подстановка делегируются ядру. В обоих режимах решение независимо верифицируется полным решением исходной системы средствами ядра.

## 2.4 Структуры данных

Данный раздел дополняет описание архитектуры со стороны организации данных. Основная задача проектирования в рамках этого контура заключается в определении форматов представления многомерных величин на каждом уровне и поиске механизмов их эффективной трансформации.

### Логическое и линейное представление матриц

Между прикладным уровнем и вычислительным ядром данные пересекают архитектурную границу, на которой меняются их требования к памяти. На прикладном уровне естественной формой матрицы является массив строк, где каждая строка - отдельный массив чисел. Такое представление удобно для ручного ввода, поэлементного обновления, отрисовки в виде таблицы и отслеживания изменений конкретной ячейки в анимации. Спецификации BLAS и LAPACK, наоборот, предполагают непрерывное размещение элементов в одном блоке памяти, поскольку именно непрерывность обеспечивает эффективное использование кэш-памяти и блочных оптимизаций.

В архитектуре проектируется два согласованных представления матрицы:

- *Логическое (двумерное)* : существует на прикладном уровне и в журнале шагов. Используется для ввода, валидации, пошаговых преобразо-

ваний и подсветки ячеек.

- *Линейное (одномерное)* : построчная (row-major) развёртка матрицы  $m \times n$  в последовательность длины  $mn$  (для квадратных СЛАУ  $m = n$ , для регрессии используются прямоугольные матрицы признаков). Формируется только в момент вызова вычислительного ядра и используется как часть строки команды, передаваемой в линейную память WebAssembly.

Векторы во всех контекстах остаются одномерными и не требуют преобразования при передаче в ядро. Преобразование двумерного представления в линейное выполняется на стороне солвера непосредственно перед формированием команды, что позволяет хранить матрицы в удобной форме на всём остальном протяжении расчёта.

### **Состав снимка шага**

Реализация алгоритмов в виде последовательности атомарных операций (см. разд. 2.3) предполагает фиксацию промежуточных состояний системы. Таким состоянием является снимок шага — самодостаточный объект данных, передаваемый от вычислительного ядра к слою визуализации. В реализации прототипа он не выделен в отдельный сериализуемый класс: снимок материализуется как согласованная пара записей в журнал шагов и обновлений визуального состояния, но логически сохраняет единый состав данных. В его состав входят:

- заголовок и описание операции в терминах линейной алгебры (например, элементарные преобразования строк или итерационное уточнение);
- текущее состояние объектов (матриц и векторов) в логическом представлении;
- метаданные подсветки: индексы ведущих, модифицируемых или целевых элементов (например, треугольных факторов);
- параметры сходимости (для итерационных методов): номер итерации, текущее приближение и норма невязки;

- строковая интерпретация команды ядра для отображения в техническом журнале.

Снимок проектируется как независимая структура, содержащая полный контекст для отрисовки. В прототипе этот контекст формируется на стороне солвера непосредственно перед вызовами журнала и анимационных примитивов, что исключает обращение компонентов визуализации к глобальному состоянию алгоритма и позволяет синхронно обновлять матричные таблицы и текстовый журнал выполнения.

## **Структуры результатов по классам задач**

В задачах СЛАУ итог расчёта формируется в журнале шагов и матричной таблице: последовательность снимков описывает выбор ведущих элементов, элиминацию и обратную подстановку, а финальный шаг включает вектор решения, факторы разложения (для прямых методов) и блок верификации с независимо полученным эталонным решением и оценкой невязки.

В нелинейных методах журнал хранит последовательность пар «приближение — норма невязки», флаг сходимости, число выполненных итераций и причину останова (достижение заданной точности либо исчерпание лимита итераций). Эта организация данных используется при формировании итерационных таблиц.

Для регрессии и аппроксимации солвер возвращает единый объект результата, в котором на верхнем уровне объединены аналитические поля (коэффициенты модели и статистические метрики качества) и поля с предвычисленными узловыми значениями и наборами данных для диаграмм. Один объект результата обслуживает несколько типов визуализаций, сохраняя аналитический контекст расчёта.

## **2.5 Интерфейсы взаимодействия**

В разделе рассматриваются пользовательский интерфейс как поверхность взаимодействия человека и системы и программные контракты между интерфейсным, прикладным и вычислительным уровнями.

### **Пользовательский интерфейс**

Пользовательский интерфейс реализован вокруг единого сценария взаимодействия для всех классов задач, включающего ввод исходных данных, запуск метода, наблюдение пошаговой эволюции внутреннего состояния алгоритма и анализ итогового результата. На интерфейсном уровне предусмотрены три типа расчётных контейнеров, ориентированных на специфические классы задач, а также навигационные страницы (главная страница и каталог методов). Несмотря на различия в средствах ввода и визуализации, расчётные контейнеры реализуют унифицированный сценарий, охватывающий валидацию данных на стороне формы, проверку состояния ядра, формирование контекста выполнения, вызов процедуры решения и рендеринг результата. Соответствие класса задачи и интерфейсных средств приведено в таблице 2.3.

| Класс задачи              | Контейнер                   | Средства ввода                                           | Средства визуализации                              |
|---------------------------|-----------------------------|----------------------------------------------------------|----------------------------------------------------|
| Линейные СЛАУ             | Контейнер СЛАУ              | Матричная сетка $n \times n$ и вектор правой части       | Подсветка ячеек расширенной матрицы и журнал шагов |
| Нелинейные системы        | Контейнер нелинейных систем | Текстовые поля уравнений и вектор начального приближения | Итерационная таблица и журнал шагов                |
| Аппроксимация и регрессия | Контейнер регрессии         | Динамическая таблица наблюдений $(x, y)$                 | Аналитические диаграммы на Canvas                  |

Таблица 2.3: Соответствие класса задачи и интерфейсных средств

Страницы-контейнеры осуществляют динамическое подключение модулей методов и передают их конфигурации в стандартный набор визуальных и управляющих компонентов. Такая организация обеспечивает расширяемость системы, при которой включение нового метода требует лишь подготовки модуля установленной структуры, его регистрации в подсистемах загрузки и навигации, а также добавления в общий каталог методов. Базовые алгоритмы работы контейнеров и общие интерфейсные средства при этом остаются неизменными.

Компоненты визуализации и журнала предоставляют солверу императивный интерфейс, включающий подсветку ячеек матрицы, поэлементное обновление значений и добавление записей с описанием шага и соответствующей BLAS-командой. Управление пошаговой визуализацией доступно пользователю через несколько предустановок скорости и операцию пропуска анимации, при которой алгоритм завершает вычисления, минуя промежуточные

кадры, и отрисовывает финальное состояние без изменения корректности результата.

Прикладные размерности задач ограничены сверху в зависимости от класса устройства (мобильный, планшет, десктоп). Категория устройства определяется по ширине окна и реактивно пересчитывается при изменении размера окна или ориентации экрана. При превышении введённой пользователем размерности значение прижимается к допустимому максимуму с информационным сообщением.

## Программные интерфейсы между уровнями

Взаимодействие интерфейсного, прикладного и вычислительного уровней опирается на три программных интерфейса. Их формальные характеристики (направление, форма обмена, режим) сведены в таблице 2.4. Содержательное описание контракта солвера и базовой модели взаимодействия страницы с ядром дано в разделе 2.3, а императивных каналов визуализации и журнала шагов — выше в блоке «Пользовательский интерфейс».

| Интерфейс               | Направление             | Форма обмена                                                                            | Режим                                     |
|-------------------------|-------------------------|-----------------------------------------------------------------------------------------|-------------------------------------------|
| Контракт солвера        | страница ↔ солвер       | солвер декларирует метаданные и операцию решения; страница передаёт контекст выполнения | декларативный контракт, асинхронный вызов |
| Командный протокол ядра | солвер ↔ ядро           | текстовая команда / текстовый ответ                                                     | синхронный, без скрытого состояния        |
| Алгоритм – визуализация | солвер → слой отрисовки | журнал снимков и анимационные примитивы                                                 | поточковый, с прерыванием                 |

Таблица 2.4: Интерфейсы взаимодействия между уровнями архитектуры

Командный протокол вычислительного ядра формализует вызовы WebAssembly-модуля. Солвер формирует строку с именем BLAS- или LAPACK-подпрограммы и её параметрами, передаёт её через инжектируемый интерфейс и получает текстовый результат. Вся логика управления памятью ядра, кодирования команды и перехвата вывода инкапсулирована в адаптере и скрыта от солвера. Текстовый формат обеспечивает прозрачность логирования и упрощает отладку взаимодействия с низкоуровневой памятью. Перечень классов операций ядра, используемых при реализации методов, приведён в таблице 2.5.

| Класс операции                                        | Назначение в контексте алгоритма                |
|-------------------------------------------------------|-------------------------------------------------|
| Поиск индекса экстремального элемента вектора         | Выбор ведущего элемента                         |
| Линейная комбинация векторов                          | Элиминация при прямом ходе                      |
| Перестановка векторов                                 | Перестановка строк при выборе ведущего элемента |
| Решение треугольной системы                           | Обратная подстановка                            |
| Полное решение СЛАУ                                   | Независимая верификация результата              |
| Решение симметрично-положительно-определённой СЛАУ    | Метод Холецкого                                 |
| QR-факторизация и применение ортогонального множителя | Метод QR                                        |
| Матрично-векторное произведение                       | Вычисление невязки и итерационных шагов         |
| Скалярное произведение и норма                        | Контроль сходимости итерационных методов        |

Таблица 2.5: Классы операций вычислительного ядра, используемые в методах

## 3 Реализация прототипа

Настоящая глава посвящена программной реализации архитектурных решений, описанных в Главе 2.

### 3.1 Подготовка вычислительного ядра

Ядро реализовано как командная оболочка над OpenBLAS и LAPACK. Основной модуль `shell_cblas.c` определяет таблицу диспетчеризации `g_commands` и единую точку входа `run_command`, которая по записи таблицы передаёт управление обработчику `cmd_*`. Состав зарегистрированных команд по категориям приведён в таблице 3.1.

| Категория    | N  | Команды                                                                                                                                                                                                                                                                                                                                                                      |
|--------------|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| BLAS Level 1 | 10 | <code>ddot</code> , <code>dnrm2</code> , <code>dasum</code> , <code>idamax</code> , <code>dscal</code> , <code>daxpy</code> , <code>dcopy</code> , <code>dswap</code> , <code>drotg</code> , <code>drot</code>                                                                                                                                                               |
| BLAS Level 2 | 7  | <code>dgemv</code> , <code>dtrsv</code> , <code>dger</code> , <code>dsymv</code> , <code>dsyr</code> , <code>dsyr2</code> , <code>dtrmv</code>                                                                                                                                                                                                                               |
| BLAS Level 3 | 5  | <code>dgemm</code> , <code>dtrsm</code> , <code>dtrmm</code> , <code>dsyrk</code> , <code>dsymm</code>                                                                                                                                                                                                                                                                       |
| LAPACK       | 17 | <code>dgesv</code> , <code>dgetrf</code> , <code>dgetrs</code> , <code>dpotrf</code> , <code>dpotrs</code> , <code>dposv</code> , <code>dgeqrf</code> , <code>dorgqr</code> , <code>dormqr</code> , <code>dgels</code> , <code>dgelsd</code> , <code>dgelss</code> , <code>dgtsv</code> , <code>dptsv</code> , <code>dgbsv</code> , <code>dsyev</code> , <code>dgesvd</code> |
| Служебные    | 2  | <code>threads</code> , <code>test</code>                                                                                                                                                                                                                                                                                                                                     |

Таблица 3.1: Категории команд вычислительного ядра

Парсеры числовых параметров (`toint`, `todbl`) и обёртки распределения памяти (`xmalloc`, `xcalloc`) при ошибке вызывают `abort_command`, который печатает диагностическое сообщение и при активном маркере `g_cmd_abort` возвращает управление в `run_command` через `longjmp`, что требует флага `SUPPORT_LONGJMP=emscripten` при сборке (табл. 3.2). Прочие ошибки обрабатываются в самих `cmd_*`-обработчиках возвратом с печатью сообщения.

Сборка автоматизирована скриптом `build.sh`: OpenBLAS компилируется под WebAssembly в режиме `TARGET=GENERIC` в однопоточной конфигурации (`USE_THREAD=0`), полученная `libopenblas.a` компоуется с `shell_cblas.c` через `emcc -O2`. Ключевые параметры компоновщика приведены в таблице 3.2.



| Параметр                     | Назначение                                                |
|------------------------------|-----------------------------------------------------------|
| INITIAL_MEMORY=67108864      | Начальный размер линейной памяти WASM (64 МБ)             |
| ALLOW_MEMORY_GROWTH=1        | Динамическое расширение памяти при необходимости          |
| INVOKE_RUN=0, EXIT_RUNTIME=0 | Ручной запуск main, рантайм не завершается после возврата |
| EXPORTED_FUNCTIONS           | _main, _run_command, _malloc, _free                       |
| EXPORTED_RUNTIME_METHODS     | stringToUTF8, lengthBytesUTF8 (доступ из JS-обёртки)      |
| SUPPORT_LONGJMP=emscripten   | Поддержка longjmp для механизма обработки ошибок          |

Таблица 3.2: Ключевые параметры компоновщика emcc

## 3.2 Подключение ядра к интерфейсному уровню

Заголовки COOP/COEP, обеспечивающие изолированный контекст безопасности и подготовку инфраструктуры к возможному переходу на pthreads-сборку OpenBLAS, проставляются на уровне сервера: их выставляют dev- и preview-серверы Vite, а также плагин раздачи артефактов из blas\_wasm/. Клиентская часть проверок изоляции не выполняет. При монтировании провайдера WasmContext вызывается loadWasm, который динамически импортирует /blas\_wasm/shell\_cblas.js и передаёт её фабрике опцию locateFile, разрешающую путь к shell\_cblas.wasm относительно /blas\_wasm/. Полученный инстанс сохраняется глобально как \_\_shellCblas, а реактивный флаг wasmReady переключается через промис.

Emscripten фиксирует ссылку на print в момент загрузки, поэтому колбэк вывода нельзя переназначать для каждого вызова ядра напрямую. Вместо этого в print один раз ставится статический прокси, делегирующий вывод текущему обработчику \_printHandler, который подменяется на время вызова ядра и затем восстанавливается. На этом механизме построена функция runBlas, представляющая собой единую точку вызова ядра из JavaScript. Её шаги приведены в таблице 3.3. Освобождение буфера и восстановление \_printHandler помещены в блок try...finally, что обеспечивает корректное состояние даже при исключении внутри ядра.

| Шаг | Действие                                                                                                                                                                   |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1   | Сохранить текущий <code>_printHandler</code> и заменить его на накопитель строк                                                                                            |
| 2   | Вычислить длину UTF-8-представления команды функцией <code>lengthBytesUTF8</code> с учётом байта нуль-терминатора                                                          |
| 3   | Выделить буфер в линейной памяти WASM через <code>_malloc</code>                                                                                                           |
| 4   | Записать строку в буфер функцией <code>stringToUTF8</code>                                                                                                                 |
| 5   | Вызвать экспортированную точку <code>_run_command</code> с указателем на буфер                                                                                             |
| 6   | В блоке <code>finally</code> освободить буфер через <code>_free</code> и восстановить прежний <code>_printHandler</code> . После выхода из блока вернуть накопленный текст |

Таблица 3.3: Последовательность операций функции `runBlas`

`WasmContext` экспортирует `useWasm`, возвращающий компонентам реактивный флаг `wasmReady`, флаг ошибки `wasmError`, функцию `loadWasm` и функцию `runBlas`. Интерфейсные элементы используют этот API без прямого обращения к `__shellCblas` или специфики работы с WebAssembly.

### 3.3 Модули-солверы

Численные методы размещены в каталоге `src/hooks/solvers/`, разделённом на подкаталоги `linear/`, `nonlinear/` и `approx/` по классу решаемой задачи. Каждый модуль экспортирует объект, следующий общей схеме контракта солвера (табл. 2.1), однако состав служебных полей частично зависит от страницы-контейнера. Номенклатура реализованных модулей по классам приведена в таблицах 3.4, 3.5 и 3.6. В столбце «Ядро» значение *да* отвечает использованию ядра в основном ходе вычислений, *верификация* отвечает контрольной проверке решения, *журнал* означает запись эквивалентной BLAS/LAPACK-команды без фактического вызова, *нет* означает отсутствие обращений к ядру. Классификация относится к математическому ходу алгоритма, тогда как на уровне интерфейса запуск всех страниц унифицированно ожидает готовности `wasmReady`.

| Метод                    | Файл                            | Ядро   | Операции ядра      |
|--------------------------|---------------------------------|--------|--------------------|
| Метод Ньютона            | <code>useNewton.js</code>       | журнал | <code>dgesv</code> |
| Метод Бройдена           | <code>useBroyden.js</code>      | журнал | <code>dgesv</code> |
| Простая итерация         | <code>useIteration.js</code>    | нет    | —                  |
| Гомотопия                | <code>useHomotopy.js</code>     | журнал | <code>dgesv</code> |
| Продолжение по параметру | <code>useContinuation.js</code> | журнал | <code>dgesv</code> |

Таблица 3.4: Модули-солверы для нелинейных систем (`nonlinear/`)

| Метод           | Файл           | Ядро        | Операции ядра                                    |
|-----------------|----------------|-------------|--------------------------------------------------|
| Метод Гаусса    | useGauss.js    | да          | idamax, dswap, daxpy, dtrsv; dgesv (верификация) |
| LU-разложение   | useLU.js       | да          | idamax, dswap, daxpy; dgesv (верификация)        |
| QR-разложение   | useQR.js       | верификация | dgesv                                            |
| Метод Холецкого | useCholesky.js | да          | dpotrf, dpotrs; dposv (верификация)              |
| Метод Якоби     | useJacobi.js   | да          | dgemv (невязка); dgesv (верификация)             |
| Метод Зейделя   | useSeidel.js   | да          | dgemv (невязка); dgesv (верификация)             |
| SOR             | useSOR.js      | да          | dgemv (невязка); dgesv (верификация)             |
| MR              | useMinRes.js   | да          | dgemv, ddot, dnrn2, daxpy; dgesv (верификация)   |
| BiCG            | useBiCG.js     | да          | dgemv, ddot; dgesv (верификация)                 |
| GMRES           | useGMRES.js    | да          | dgemv, dnrn2; dgesv (верификация)                |

Таблица 3.5: Модули-солверы для линейных СЛАУ (linear/)

| Метод                    | Файл                   | Ядро | Операции ядра              |
|--------------------------|------------------------|------|----------------------------|
| Линейная регрессия       | useLinearRegression.js | да   | dgesv; dgemv (верификация) |
| Полиномиальная регрессия | usePolyRegression.js   | да   | dgesv; dgemv (верификация) |
| Метод наим. квадратов    | useLeastSquares.js     | да   | dgesv; dgemv (верификация) |
| RBF-интерполяция         | useRBF.js              | да   | dgesv; dgemv (верификация) |
| Сплайн                   | useSpline.js           | да   | dgesv                      |

Таблица 3.6: Модули-солверы для аппроксимации и регрессии (approx/)

Модули солверов перечислены в словарях `solverConfigs`, `nonlinearConfigs` и `approxConfigs`, которые сопоставляют ключу маршрута динамический `import()` соответствующего файла. Помимо перехода на маршрут, модуль может прогреваться через `preloadAlgorithm` при наведении, фокусе или открытии модального окна на странице алгоритмов.

Часть этапов оставлена на JavaScript сознательно. Прямая и обратная подстановки в LU и преобразования Хаусхолдера в QR выполняются на клиенте ради развёрнутого пошагового журнала и поэтапной визуализации. В нелинейных методах (Ньютон, Бroyден, гомотопия, продолжение по параметру) возникающие линейные подсистемы решаются JS-процедурой `luSolve`, а в журнал помещается эквивалентная команда `dgesv` без вызова ядра. В BiCG произведение  $A^T \tilde{r}$  вычисляется JS-функцией `matTVec`, тогда как соответствующий вызов `dgemv` журналируется ради единообразия протоколирования.

## 3.4 Режим рациональных дробей

Режим реализован классом `Frac` в `solverUtils.js`. Это пара полей `n` (числитель) и `d` (знаменатель), хранящих целые значения. Конструктор приводит знаменатель к положительному значению и сокращает дробь по наибольшему общему делителю, вычисляемому алгоритмом Евклида. Полный набор операций класса приведён в таблице 3.7.

| Метод                                       | Операция     | Возвращаемый результат              |
|---------------------------------------------|--------------|-------------------------------------|
| <code>add(o), sub(o), mul(o), div(o)</code> | арифметика   | новый объект <code>Frac</code>      |
| <code>neg(), abs()</code>                   | унарные      | новый объект <code>Frac</code>      |
| <code>isZero()</code>                       | предикат     | булево значение                     |
| <code>absVal()</code>                       | $ n/d $      | число с плавающей запятой           |
| <code>toNumber()</code>                     | $n/d$        | число с плавающей запятой           |
| <code>toString()</code>                     | сериализация | строка $n/d$ , либо $n$ при $d = 1$ |

Таблица 3.7: Операции класса `Frac`

Переключение режимов опирается на предикат `isAllInt(A, b)`. При целочисленных  $A$  и  $b$  входные данные конвертируются функциями `toFracMatrix` и `toFracVec`. В методах Гаусса и LU алгоритм солвера сохраняется единой функцией, а различаются лишь определения арифметических операций, выбираемые тернарной развязкой по флагу `exact` (в типичной форме `const add = exact ? (a,b) => a.add(b) : (a,b) => a + b`). Такой приём позволяет обоим режимам использовать одну и ту же последовательность шагов визуализации. В QR-разложении сами матрицы  $Q$  и  $R$  строятся в числах с плавающей точкой, так как операции с квадратными корнями выходят за поле рациональных чисел. Ветвь `Frac` в этом методе применяется только к решению исходной СЛАУ гауссовой элиминацией.

Типографское отображение дробей в HTML выполняется утилитой `fmtNumHtml`. При  $d \neq 1$  формируется разметка `<span class="frac"><sup>n</sup>&frasl;<sub>d</sub></span>` с учётом знака. Верификация решения сводится к численному сравнению. Значения переводятся методом `toNumber`, после чего полученный вектор сопоставляется с эталоном `dgesv` по допуску  $10^{-4}$ . Таким образом, подтверждается практическое совпадение с численным решением, а не символьное равенство в поле  $\mathbb{Q}$ . Класс `Frac` не является универсальной библиотекой безопасной рациональной арифметики. Конструктор не проверяет случай  $d = 0$ , а корректность деления

и отсутствие нулевых опорных элементов обеспечиваются внешними проверками на уровне солверов.

## 3.5 Слой пошаговой визуализации и ввода

Компоненты, обслуживающие пошаговое решение, реализуют пошаговую DOM-визуализацию матричных алгоритмов и журнал шагов и построены как React-компоненты с императивным интерфейсом через `ref`. Компоненты `Visualization` и `StepLog` отвечают за отображение результата, `MatrixInput` — за подготовку входных данных. Иначе говоря, интерфейс «алгоритм-визуализация» из Главы 2 реализован набором императивных примитивов, через которые солвер материализует очередной снимок шага. Через `forwardRef` и `useImperativeHandle` экспортируются, в частности, методы, перечисленные в таблице 3.8.

| Компонент     | Метод                                                                              | Назначение                                                                                                                                                               |
|---------------|------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Visualization | <code>show, hide</code>                                                            | показ и скрытие секции                                                                                                                                                   |
|               | <code>getSpeed</code>                                                              | текущая скорость анимации (мс/шаг)                                                                                                                                       |
|               | <code>setStatus, setOpLabel</code>                                                 | заголовок и метка текущей операции                                                                                                                                       |
|               | <code>setContainerHTML, appendHTML</code>                                          | полная или инкрементальная замена содержимого таблицы                                                                                                                    |
|               | <code>querySelector, querySelectorAll</code>                                       | выборка элементов в области таблицы                                                                                                                                      |
|               | <code>enableBacksub, disableBacksub</code>                                         | регистрация и отключение второй секции визуализации обратной подстановки                                                                                                 |
| StepLog       | <code>setBacksubStatus, setBacksubContainerHTML, backsubQuerySelector</code> и др. | управление содержимым секции обратной подстановки                                                                                                                        |
|               | <code>clear</code>                                                                 | очистка журнала                                                                                                                                                          |
|               | <code>addStep</code>                                                               | добавление скрытого шага с возвратом ссылки на DOM-элемент                                                                                                               |
| MatrixInput   | <code>showStep</code>                                                              | асинхронный показ шага через паузу <code>stepDelay</code> (Promise)                                                                                                      |
|               | <code>buildGrid</code>                                                             | построение сетки <code>&lt;input&gt;</code> -полей размером $n \times n$                                                                                                 |
|               | <code>readInput</code>                                                             | чтение и валидация <code>n, A, b</code> ; дополнительные поля <code>extra</code> передаются как строки; результат <code>{ n, A, b, extra }</code> либо <code>null</code> |
|               | <code>loadExample</code>                                                           | подстановка демонстрационных данных из контракта                                                                                                                         |

Таблица 3.8: Императивные методы компонентов визуализации и ввода

Скорость анимации задаётся пользователем в `Visualization` выпадающим списком со значениями 3200, 1600 и 800 мс на шаг. Атомарные операции над таблицей собраны в `solverUtils.js: renderImat` формирует HTML расширенной матрицы с атрибутами `data-row/data-col` и передаёт его в `viz.setContainerHTML`; `highlightCell`, `highlightRow`, `updateCell` и `clearHighlights` работают с уже созданными ячейками че-

рез `querySelector/querySelectorAll` объекта `viz`. Это позволяет повторно использовать функции и в основной таблице, и в секции обратной подстановки, которая включается для сценария метода Гаусса.

В компоненте `StepLog` метод `addStep` создаёт скрытый `<div>` с заголовком, описанием, фрагментом таблицы и, при необходимости, диагностической или низкоуровневой командой, а `showStep` ожидает `stepDelay` миллисекунд и снимает у этого элемента класс `step--hidden`, делая шаг видимым. Поскольку `showStep` возвращает `Promise`, темп журнала контролируется на стороне солвера через `await`. Между шагами в интерактивных линейных и нелинейных солверах дополнительно вставляется ожидание `animSleep(viz.getSpeed(), skipRef)`, прерываемое флагом `skipRef`, что позволяет пользователю мгновенно завершить анимацию и сразу увидеть итоговый результат.

## 3.6 Страницы-контейнеры

Маршруты страниц-контейнеров зарегистрированы в `src/App.jsx`. Сами страницы загружаются через `React.lazy` внутри `<Suspense>`, что обеспечивает разбиение бандла по маршрутам.

Унифицированный сценарий из разд. 2.5 материализован на странице последовательностью шагов: чтение входных данных, валидация, проверка `wasmReady`, формирование контекста выполнения `ctx` и вызов `config.solve(ctx)`. Для нелинейных страниц проверка `wasmReady` носит инфраструктурный характер, так как сами вычисления остаются на JavaScript-уровне. Состав контекста зависит от типа страницы и приведён в таблице 3.9: `SolverPage` и `NonlinearSolverPage` передают полный набор полей, `RegressionPage` ограничивается `data`, `runBlas` и `stepLog`, поскольку в нём отсутствует интерактивная матричная визуализация и пропуск анимации.

Адаптивный лимит размерности из разд. 2.5 реализован хуком `useViewport`: категория устройства определяется по порогам 600 и 1024 пикселей, а соответствующие лимиты возвращают функции `maxMatrixSize`, `maxNonlinearSize` и `maxRegressionFeatures` (таблица 3.10). Страница применяет возвращённое значение при построении сетки `MatrixInput` и при ограничении числа признаков в таблице наблюдений.

| Поле      | Источник                                              | Назначение                                    | Стр.     |
|-----------|-------------------------------------------------------|-----------------------------------------------|----------|
| data      | readInput (SLV); локальное состояние формы (NLS, REG) | входные данные задачи                         | все      |
| runBlas   | хук useWasm                                           | вызов вычислительного ядра                    | все      |
| stepLog   | ref компонента StepLog                                | журнал шагов                                  | все      |
| viz       | ref компонента Visualization                          | императивный интерфейс матричной визуализации | SLV, NLS |
| skipRef   | локальный useRef страницы                             | флаг пропуска анимации                        | SLV, NLS |
| wasmReady | хук useWasm                                           | признак готовности ядра                       | SLV, NLS |

Таблица 3.9: Состав контекста выполнения ctx

| Класс задачи                 | Mobile<br>(≤ 600 px) | Tablet<br>(≤ 1024 px) | Desktop<br>(иначе) |
|------------------------------|----------------------|-----------------------|--------------------|
| Линейные СЛАУ (порядок)      | 5                    | 8                     | 11                 |
| Нелинейные системы (порядок) | 4                    | 7                     | 10                 |
| Регрессия (число признаков)  | 3                    | 4                     | 5                  |

Таблица 3.10: Адаптивные ограничения на размерность задач

SolverPage обслуживает линейные методы. На монтировании страница загружает контракт через configLoader, монтирует MatrixInput, Visualization и StepLog с пропсами из контракта (prefix, defaultSize, matrixLabel, extraParams, stepDelay). Перед запуском солвера страница читает данные методом matrixRef.current.readInput(), очищает журнал и отключает секцию обратной подстановки.

NonlinearSolverPage формирует  $n$  текстовых полей для уравнений (синтаксис вида  $x_1^2 + x_2^2 - 4$ ) и поле начального приближения  $x_0$ , передавая собранные строки выражений и стартовый вектор в составе контекста выполнения солверу нелинейной системы.

RegressionPage формирует динамическую таблицу наблюдений со строками вида  $(x_1, \dots, x_m, y)$ , причём число признаков  $m$  выбирается в пределах таблицы 3.10. Перед запуском страница отбрасывает строки с неполными или нечисловыми значениями и проверяет минимальный объём выборки: если конфигурация солвера определяет метод config.minPoints(m), используется он, иначе применяется порог по умолчанию  $m + 1$ . Полученный от солвера объект результата передаётся модулю итоговых диаграмм (см. разд. 3.7).

## 3.7 Модуль итоговых диаграмм

Графический модуль регрессионных и аппроксимационных методов реализован без сторонних библиотек в файле src/utils/chartRenderer.js.



Графики рисуются средствами стандартного 2D Canvas API, а вспомогательные элементы интерфейса создаются обычными DOM-элементами. `createCanvas(w, h, colors)` создаёт элемент с учётом `window.devicePixelRatio`. Физический размер canvas увеличивается в `dpr` раз, после чего контекст немедленно масштабируется обратным вызовом `ctx.scale(dpr, dpr)`, что гарантирует чёткость линий и текста на дисплеях высокой плотности.

Точка входа `renderCharts(container, charts)` получает контейнер и объект результата солвера. Далее функция выбирает палитру через `getColors()`, вычисляет размеры canvas по текущей ширине контейнера и вызывает специализированные функции отрисовки для тех верхнеуровневых полей, которые присутствуют в объекте результата. Список поддерживаемых типов приведён в таблице 3.11.

| Тип                        | Функция                               | Применение                                                    |
|----------------------------|---------------------------------------|---------------------------------------------------------------|
| Карточка с формулой модели | <code>renderEquationCard</code>       | линейная и полиномиальная регрессии, МНК                      |
| Таблица числовых метрик    | <code>renderMetrics</code>            | все регрессионные и аппроксимационные солверы                 |
| Рассеяние с моделью        | <code>renderScatterChart</code>       | одномерные солверы ( $m = 1$ )                                |
| Частная зависимость        | <code>renderPartialDependence</code>  | линейная и полиномиальная регрессии                           |
| Предсказание против факта  | <code>renderPredVsActual</code>       | линейная и полиномиальная регрессии, МНК, RBF, сплайн         |
| Остатки (рассеяние)        | <code>renderResidualsScatter</code>   | линейная и полиномиальная регрессии, МНК, RBF, сплайн         |
| Гистограмма остатков       | <code>renderResidualHistogram</code>  | линейная и полиномиальная регрессии, МНК                      |
| Коэффициенты модели        | <code>renderCoefficientsChart</code>  | линейная и полиномиальная регрессии, МНК, RBF, multi-D сплайн |
| Тепловая карта корреляций  | <code>renderCorrelationHeatmap</code> | линейная и полиномиальная регрессии при $m > 1$               |

Таблица 3.11: Типы диаграмм графического модуля

Модуль ожидает на вход объект результата солвера, состав которого описан в разд. 2.4. Наличие или отсутствие отдельных верхнеуровневых полей этого объекта непосредственно определяет итоговый набор диаграмм. Повторный вызов `renderCharts` с новым объектом результата приводит к полной перерисовке контейнера без повторного запуска вычислений. Сам модуль не подписывается на события изменения размера окна или смены темы и выполняет перерисовку только при явном вызове со стороны страницы-потребителя.



## 4 Описание эксперимента и методика оценки платформы

В главе формализована воспроизводимая методика оценки платформы на основе ключевых метрик, тестовых задач и фиксированной конфигурации среды. Анализ строится по двум независимым контурам. Межбраузерный контур измеряет характеристики системы в различных браузерах, а внешний контур сопоставляет платформу с альтернативными вычислительными средами и онлайн-солверами.

### 4.1 Межбраузерный эксперимент

Межбраузерный эксперимент направлен на оценку устойчивости работы разрабатываемой платформы при смене браузерного движка. В его рамках одна и та же версия системы запускается в Chromium, Firefox и WebKit на фиксированной аппаратной базе и при одинаковом наборе сценариев.

| Блок                         | Сценарий                                                                      | Метрики                                                   | Единицы измерения                | Смысл                                                                           |
|------------------------------|-------------------------------------------------------------------------------|-----------------------------------------------------------|----------------------------------|---------------------------------------------------------------------------------|
| Загрузка и старт             | Холодное открытие главной страницы и первого solver-маршрута                  | wasm startup, FCP, load                                   | мс, мс, мс                       | Скорость инициализации ядра и вывода стартового интерфейса                      |
| Навигация и UI               | Переходы между маршрутами, загрузка примеров, запуск решателя через интерфейс | routes ok, feature support, console errors, layout stable | %, %, шт., %                     | Целостность пользовательского сценария и межбраузерная совместимость интерфейса |
| Линейные методы              | Решение СЛАУ прямыми и итерационными методами на фиксированных входных данных | median solve time, relative error, residual, convergence  | мс, безразм., безразм., %        | Скорость и численная корректность решения линейных задач                        |
| Нелинейные методы            | Решение систем нелинейных уравнений при различных стартовых приближениях      | median solve time, iterations, residual, convergence rate | мс, шт., безразм., %             | Скорость сходимости и устойчивость к выбору стартового приближения              |
| Аппроксимация и интерполяция | Регрессия и интерполяция на наборах точек разного объема                      | median fit time, $R^2$ , RMSE, max  residual              | мс, безразм., безразм., безразм. | Скорость построения модели и точность приближения                               |
| Визуализация                 | Пошаговое воспроизведение решения и анимация состояний                        | avg FPS, p1 FPS, drops < 30 FPS                           | кадр/с, кадр/с, шт.              | Плавность пошагового режима и устойчивость анимации                             |

Таблица 4.1: Межбраузерный эксперимент

Рассмотрим формальные определения метрик по каждому блоку:

## Загрузка и старт

Пусть  $t_{\text{nav}}$  — момент начала навигации,  $t_{\text{fcp}}$  — момент первой содержательной отрисовки,  $t_{\text{load}}$  — момент срабатывания события `window.load`, а  $t_{\text{wasm}}$  — момент перехода флага `wasmReady` в значение `true`. Тогда

$$\begin{aligned} \text{FCP} &= t_{\text{fcp}} - t_{\text{nav}}, & \text{load} &= t_{\text{load}} - t_{\text{nav}}, \\ \text{wasm startup} &= t_{\text{wasm}} - t_{\text{nav}}. \end{aligned}$$

## Навигация и UI

Пусть  $R$  — множество маршрутов сценария,  $R_{\text{ok}} \subseteq R$  — подмножество маршрутов, страница которых смонтировалась без необработанной ошибки. Пусть  $F$  — множество проверяемых функций интерфейса,  $F_{\text{ok}} \subseteq F$  — поддерживаемых в данном браузере. Пусть  $E$  — число записей уровня `error` в консоли, накопленных за сценарий, и CLS — интегральный показатель Cumulative Layout Shift, фиксируемый PerformanceObserver. Тогда

$$\begin{aligned} \text{routes ok} &= \frac{|R_{\text{ok}}|}{|R|}, & \text{feature support} &= \frac{|F_{\text{ok}}|}{|F|}, \\ \text{console errors} &= E, & \text{layout stable} &= \max(0, 1 - \text{CLS}). \end{aligned}$$

## Линейные методы

Для  $K$  повторных запусков  $i = 1, \dots, K$  обозначим  $T_i$  — время решения СЛАУ  $Ax = b$  солвером,  $\hat{x}_i$  — полученное приближение,  $x_*$  — эталонное решение, найденное вызовом `dgesv` на тех же данных. Пусть  $K_*$  — число запусков, у которых верификация прошла с заданным допуском  $\varepsilon$ . Тогда

$$\begin{aligned} \text{median solve time} &= \text{med}_i T_i, & \text{relative error} &= \text{med}_i \frac{\|\hat{x}_i - x_*\|_2}{\|x_*\|_2}, \\ \text{residual} &= \text{med}_i \frac{\|A\hat{x}_i - b\|_2}{\|b\|_2}, & \text{convergence} &= \frac{K_*}{K}. \end{aligned}$$

## Нелинейные методы

Для нелинейной системы  $F(x) = 0$  выполняется серия из  $K$  запусков с различными стартовыми приближениями  $x_i^{(0)}$ . Пусть  $T_i$  — время решения,  $N_i$  — число итераций до сходимости (или до исчерпания лимита),  $\hat{x}_i$  — финальное приближение, а  $K_*$  — число стартов, для которых  $\|F(\hat{x}_i)\|_2 \leq \varepsilon$ . Тогда

$$\begin{aligned}\text{median solve time} &= \text{med}_i T_i, & \text{iterations} &= \text{med}_i N_i, \\ \text{residual} &= \text{med}_i \|F(\hat{x}_i)\|_2, & \text{convergence rate} &= \frac{K_\star}{K}.\end{aligned}$$

## Аппроксимация и интерполяция

Пусть имеется выборка  $\{(x_i, y_i)\}_{i=1}^N$ ,  $\hat{y}_i$  — предсказание построенной модели в точке  $x_i$ ,  $\bar{y} = N^{-1} \sum_i y_i$  — среднее значение отклика,  $T_j$  — время построения модели в  $j$ -м запуске. Тогда

$$\begin{aligned}\text{median fit time} &= \text{med}_j T_j, & R^2 &= 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2}, \\ \text{RMSE} &= \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}, & \max |\text{residual}| &= \max_{i=1, \dots, N} |y_i - \hat{y}_i|.\end{aligned}$$

## Визуализация

Пусть в течение анимационного сценария зафиксирована последовательность интервалов между кадрами  $\{\Delta t_k\}_{k=1}^M$  (мс), а мгновенная частота кадров вычисляется как  $f_k = 1000/\Delta t_k$ . Тогда

$$\begin{aligned}\text{avg FPS} &= \frac{1}{M} \sum_{k=1}^M f_k, & \text{p1 FPS} &= Q_{0,01}(\{f_k\}), \\ \text{drops} < 30 \text{ FPS} &= |\{k : f_k < 30\}|,\end{aligned}$$

где  $Q_{0,01}$  — эмпирическая квантиль порядка 0,01 распределения  $\{f_k\}$ .

## 4.2 Метрики сравнения с внешними платформами

Внешнее исследование ориентировано на сопоставление проектируемой платформы с релевантными веб-системами, классифицируемыми как браузерные вычислительные среды, электронные образовательные платформы математического профиля, онлайн-солверы и инструменты научной визуализации. Данные границы декомпозиции позволяют комплексно оценить позиционирование разрабатываемого решения в ряду существующих аналогов, учитывая не только показатели вычислительной эффективности, но и функ-

циональную наполнение, модель исполнения, а также архитектуру и удобство пользовательского интерфейса.

На первом этапе разрабатываемая платформа сопоставляется по единственной количественной метрике `reported_speedup` с работами Bhonsle et al. [4], Yan et al. [9] и Wang [12]. Это позволяет проверить, согласуются ли значения, полученные в данной системе, с уровнями, зафиксированными в опубликованных Wasm-исследованиях.

### **reported\_speedup**

Пусть  $T_{JS}$  и  $T_{Wasm}$  — времена исполнения JavaScript- и WebAssembly-вариантов одного и того же вычисления. Тогда

$$\text{reported\_speedup} = \frac{T_{JS}}{T_{Wasm}}.$$

На втором этапе платформа сопоставляется с интерактивными математическими системами и онлайн-солверами, такими как GeoGebra Classic [17], matrixcalc.org [18], WolframAlpha [19], Octave Online [20] и WebTigerPython [8], по следующим метрикам:

| Поле                    | Как собирается                               | Как оценивается                                | Что показывает                                        |
|-------------------------|----------------------------------------------|------------------------------------------------|-------------------------------------------------------|
| <code>lcp_sec</code>    | Lighthouse, метрика Largest Contentful Paint | время в секундах (меньше — лучше)              | Скорость вывода основного контента стартовой страницы |
| <code>perf_score</code> | Lighthouse, категория Performance            | численная оценка $\in [0, 1]$ (больше — лучше) | Интегральный профиль стартовой web-производительности |
| <code>a11y_score</code> | Lighthouse, категория Accessibility          | численная оценка $\in [0, 1]$ (больше — лучше) | Базовая доступность пользовательского интерфейса      |

Таблица 4.2: Поля сравнения на втором этапе

Рассмотрим формальные определения представленных метрик:

### **lcp\_sec**

Lighthouse регистрирует события `largest-contentful-paint` через `PerformanceObserver` и фиксирует момент  $t_{lcp}$  отрисовки наибольшего содержательного элемента (изображения, видеопостера или текстового блока) в начальном viewport. Пусть  $t_{nav}$  — момент начала навигации (в мс). Тогда

$$\text{lcp\_sec} = \frac{t_{lcp} - t_{nav}}{1000}.$$

### **perf\_score**

Метрика вычисляется как взвешенное среднее нормированных оце-

нок пяти core-метрик категории Performance: First Contentful Paint (FCP), Speed Index (SI), Largest Contentful Paint (LCP), Total Blocking Time (TBT) и Cumulative Layout Shift (CLS). Каждое измеренное значение  $m_i$  преобразуется в нормированную оценку  $\sigma_i(m_i) \in [0, 1]$  по логнормальному распределению референсных значений, после чего агрегируется с весами  $w_i$ :

$$\text{perf\_score} = \sum_{i=1}^5 w_i \sigma_i(m_i), \quad \sum_{i=1}^5 w_i = 1.$$

### **a11y\_score**

Метрика рассчитывается как взвешенное отношение пройденных проверок движка axe-core. Пусть  $A$  — множество применимых проверок,  $w_a \in \{1, 3, 7, 10\}$  — вес проверки  $a$  согласно её влиянию на пользователя,  $p_a \in \{0, 1\}$  — бинарный результат: 1 — проверка пройдена, 0 — провалена. Тогда

$$\text{a11y\_score} = \frac{\sum_{a \in A} w_a p_a}{\sum_{a \in A} w_a}.$$

## 5 Результаты эксперимента и сравнительной оценки платформы

В данной главе приведены фактические результаты эксперимента, описанного в главе 4, для текущего прототипа, а также результаты сравнительной оценки по отношению к внешним ориентирам.

### 5.1 Результаты межбраузерного эксперимента

Результаты блока загрузки и старта приведены в таблице 5.1.

| Браузер  | Wasm startup, мс | FCP, мс | Load, мс |
|----------|------------------|---------|----------|
| Chromium | 28.3             | 211.7   | 115.8    |
| Firefox  | 16.8             | 223.4   | 242.2    |
| WebKit   | 14.4             | 141.0   | 93.4     |

Таблица 5.1: Метрики блока загрузки и старта по браузерам

**Вывод по таблице:** время старта во всех браузерах удерживается в десятках миллисекунд; лучшие значения по всем трём метрикам показывает WebKit, Firefox существенно сократил FCP и Load.

Результаты блока навигации и пользовательского интерфейса приведены в таблице 5.2.

| Браузер  | Routes ok, % | Feature support, % | Console errors | Layout stable, % |
|----------|--------------|--------------------|----------------|------------------|
| Chromium | 100.0        | 100.0              | 0              | 100.0            |
| Firefox  | 100.0        | 100.0              | 0              | 100.0            |
| WebKit   | 100.0        | 100.0              | 0              | 100.0            |

Таблица 5.2: Метрики блока навигации и пользовательского интерфейса по браузерам

**Вывод по таблице:** сценарий полностью воспроизводится во всех

трёх браузерах — маршруты, функции интерфейса и адаптивная вёрстка работают без ошибок и переполнений.

Результаты блока линейных методов приведены в таблице 5.3.

| Браузер  | Median solve time, мс | Median relative error  | Median residual        | Convergence, % |
|----------|-----------------------|------------------------|------------------------|----------------|
| Chromium | 0.08                  | $1.255 \cdot 10^{-10}$ | $1.197 \cdot 10^{-15}$ | 100.0          |
| Firefox  | 0.08                  | $1.255 \cdot 10^{-10}$ | $1.197 \cdot 10^{-15}$ | 100.0          |
| WebKit   | 0.10                  | $1.255 \cdot 10^{-10}$ | $1.197 \cdot 10^{-15}$ | 100.0          |

Таблица 5.3: Метрики блока линейных методов по браузерам

**Вывод по таблице:** линейный блок демонстрирует 100% сходимость во всех браузерах при ошибке и невязке практически машинного масштаба. Разброс по времени составляет десятые доли миллисекунды. Контур численно корректен и межбраузерно стабилен.

Результаты блока нелинейных методов приведены в таблице 5.4.

| Браузер  | Median solve time, мс | Iterations | Median residual        | Convergence rate, % |
|----------|-----------------------|------------|------------------------|---------------------|
| Chromium | 0.02                  | 12         | $3.607 \cdot 10^{-11}$ | 100.0               |
| Firefox  | 0.04                  | 12         | $3.607 \cdot 10^{-11}$ | 100.0               |
| WebKit   | 0.02                  | 12         | $3.607 \cdot 10^{-11}$ | 100.0               |

Таблица 5.4: Метрики блока нелинейных методов по браузерам

**Вывод по таблице:** нелинейный блок воспроизводится во всех движках. Медианная невязка имеет порядок  $10^{-11}$ , сходимость составляет 100%. Время выполнения находится в пределах единиц миллисекунд, узких мест по производительности не наблюдается.

Результаты блока аппроксимации и интерполяции приведены в таблице 5.5.

| Браузер  | Median fit time, мс | $R^2$    | RMSE                  | Max  residual         |
|----------|---------------------|----------|-----------------------|-----------------------|
| Chromium | 0.07                | 1.000000 | $7.780 \cdot 10^{-5}$ | $1.443 \cdot 10^{-4}$ |
| Firefox  | 0.14                | 1.000000 | $7.780 \cdot 10^{-5}$ | $1.443 \cdot 10^{-4}$ |
| WebKit   | 0.14                | 1.000000 | $7.780 \cdot 10^{-5}$ | $1.443 \cdot 10^{-4}$ |

Таблица 5.5: Метрики блока аппроксимации и интерполяции по браузерам

**Вывод по таблице:** качественные метрики совпадают во всех браузерах ( $R^2 = 1.0$ , ошибки не выше  $10^{-4}$ ). Медианное время подгонки составляет десятые доли миллисекунды.

Метрики блока визуализации приведены в таблице 5.6.

| Браузер  | Avg FPS | p1 FPS | Drops < 30 FPS |
|----------|---------|--------|----------------|
| Chromium | 60.0    | 56.8   | 0              |
| Firefox  | 60.0    | 56.8   | 0              |
| WebKit   | 59.8    | 26.3   | 98             |

Таблица 5.6: Метрики блока визуализации по браузерам

**Вывод по таблице:** средний FPS близок к 60 во всех браузерах. В Chromium и Firefox просадок нет, а WebKit сохраняет хвост задержек ниже 30 FPS.

## 5.2 Результаты сопоставления с внешними платформами

На рис. 5.1 приведено сопоставление по метрике `reported_speedup`. Для разработанной платформы значение получено на прямом `dense-solve` сравнении `dgesv` (Wasm) против `luSolve` (JS) при  $n = 100$ . Для внешних работ взяты опубликованные оценки на собственных сценариях.

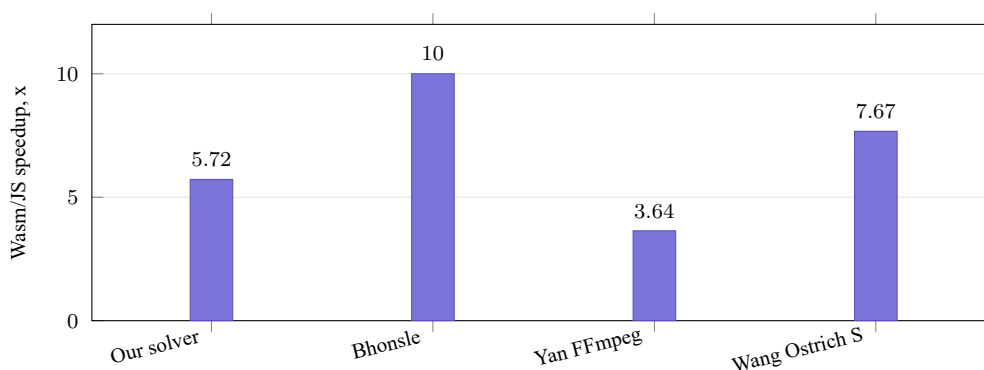


Рис. 5.1: Сопоставление WebAssembly и JavaScript по метрике `reported_speedup`



**Вывод по графику:** во всех включённых сценариях WebAssembly даёт выигрыш относительно JavaScript. Полученное для платформы значение  $5.72\times$  согласуется по порядку величины с внешними ориентирами и наиболее близко к solver-level результатам Bhonsle et al.

На рис. 5.2 приведено сопоставление стартовой web-производительности по метрике `lcp_sec` для платформ с воспроизводимо зафиксированным значением.

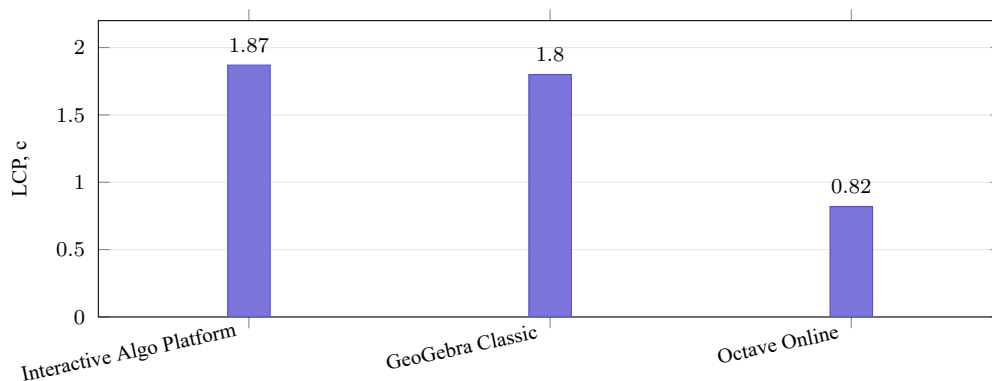


Рис. 5.2: Сравнение стартовой web-производительности по метрике LCP

**Вывод по графику:** платформа находится на одном уровне с GeoGebra Classic и уступает Octave Online, относящемуся к иной модели исполнения с удалённым backend.

На рис. 5.3 приведено сопоставление по интегральной Lighthouse-метрике `perf_score`.

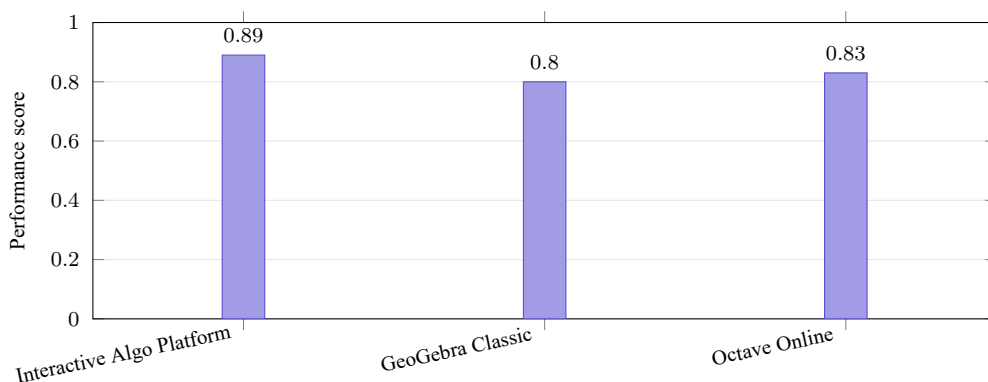


Рис. 5.3: Сравнение внешних платформ по интегральной метрике Lighthouse Performance score

**Вывод по графику:** платформа превосходит оба внешних ори-

ентира, что подтверждает конкурентоспособный профиль стартовой web-производительности.

На рис. 5.4 приведено сопоставление по метрике доступности интерфейса `a11y_score`.

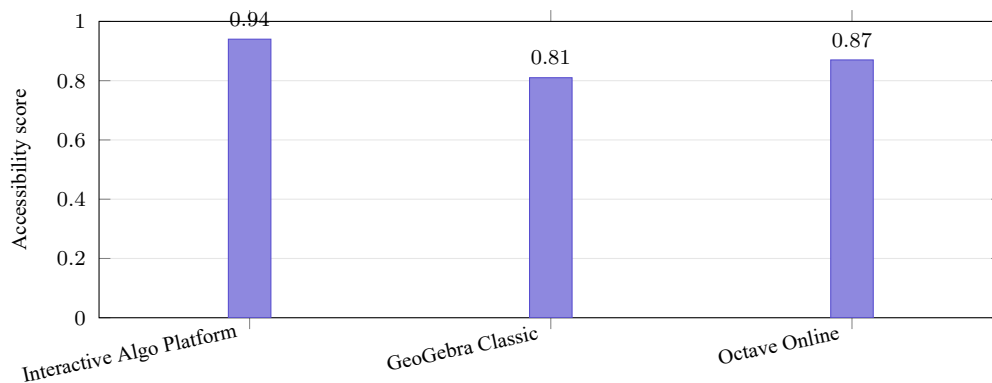


Рис. 5.4: Сравнение внешних платформ по метрике Accessibility score

**Вывод по графику:** платформа превосходит оба доступных ориентира по доступности интерфейса, что подтверждает высокий уровень соответствия пользовательских элементов рекомендациям accessibility.

## Выводы

Анализ существующих инструментов численного анализа показал, что ниша полностью клиентских веб-платформ с пошаговой визуализацией и автономным вычислительным контуром остаётся слабо заполненной.

Исследование WebAssembly подтвердило применимость технологии как основы вычислительного ядра. Связка Emscripten и типизированных массивов JavaScript обеспечивает однократную компиляцию нативного кода и доступ к линейной памяти WebAssembly из JavaScript без копирования. Это снимает основные ограничения JavaScript-реализаций линейной алгебры, отмеченные в работах Bhonsle et al. [4] и Poltavskyi [5].

Спроектирована и реализована трёхслойная архитектура прототипа с унифицированным контрактом солвера и командным протоколом обращения к вычислительному ядру. Платформа охватывает двадцать методов решения линейных, нелинейных систем и задач аппроксимации, её слоистая организация изолирует вычисления от интерфейса и обеспечивает единообразное подключение новых методов без изменения ядра.

Разработана двухконтурная методика оценки и проведён вычислительный эксперимент. Внутренний контур в Chromium, Firefox и WebKit подтвердил воспроизводимость базового сценария, 100% сходимость линейного и нелинейного блоков, медианную невязку порядка  $10^{-11}$  и качество аппроксимации  $R^2 = 1.0$ . В профильном dense-solve сценарии измерен устойчивый выигрыш WebAssembly относительно JavaScript в  $5,72\times$ , согласующийся по порядку величины с внешними ориентирами. Внешний контур показал конкурентоспособность платформы по стартовой web-производительности и доступности интерфейса.

# Заключение

В дипломной работе достигнута поставленная цель и решены все запланированные задачи. Разработан и протестирован прототип веб-платформы интерактивной визуализации и анализа многомерных численных алгоритмов [28], объединяющий вычислительное ядро на базе WebAssembly со средствами пошагового отображения вычислительного процесса.

Полученные результаты показывают, что разрыв между потребностью в доступных инструментах численного анализа и возможностями существующих платформ может быть сокращён переносом сложившихся численных библиотек в браузерную среду. Локальное исполнение в сочетании с пошаговой визуализацией обеспечивает прямое взаимодействие пользователя с вычислительным процессом без зависимости от удалённого бэкенда и при сохранении конкурентоспособной производительности.

Дальнейшая работа имеет несколько направлений. Первое — оптимизация визуализации в WebKit и расширение перечня категорий численных методов, а также числа методов внутри категорий. Второе — развитие поддержки разреженных и плохо обусловленных систем и исследование многопоточного режима OpenBLAS. Третье — расширение матрицы тестируемых браузеров с включением Edge и мобильных браузеров и автоматизация контроля производительности в CI.

## Список литературы

- [1] Chattopadhyay S., Prasad I., Henley A. Z., Sarma A., Barik T. What's Wrong with Computational Notebooks? Pain Points, Needs, and Design Opportunities // Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems (CHI). — New York: ACM, 2020. — P. 1–12. — DOI: 10.1145/3313831.3376729.
- [2] Yohannes A., Chen H.-L. GeoGebra in Mathematics Education: A Systematic Review of Journal Articles Published from 2010 to 2020 // Interactive Learning Environments. — 2023. — Vol. 31, No. 9. — P. 5682–5697. — DOI: 10.1080/10494820.2021.2016861.
- [3] Haas A., Rossberg A., Schuff D. L., Titzer B. L., Gohman D., Wagner L., Zakai A., Bastien JF, Holman M. Bringing the Web up to Speed with WebAssembly // Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI). — New York: ACM, 2017. — P. 185–200. — DOI: 10.1145/3062341.3062363.
- [4] Bhonsle A., Patil V., Valkunde T., Lotlikar T. Linear Algebra in the Browser powered by WebAssembly // International Conference for Advancement in Technology (ICONAT). — IEEE, 2022. — P. 1–6. — DOI: 10.1109/ICONAT53423.2022.9725939.
- [5] Poltavskyi D. Integration of WebAssembly in Performance-critical Web Applications // American Scientific Research Journal for Engineering, Technology, and Sciences. — 2025.
- [6] Usher W., Pascucci V. Interactive Visualization of Terascale Data in the Browser: Fact or Fiction? // IEEE 10th Symposium on Large Data Analysis and Visualization (LDAV). — IEEE, 2020. — P. 27–36. — DOI: 10.1109/LDAV51489.2020.00010.
- [7] Rein H. Real time, cross platform visualizations with zero dependencies for the N-body package REBOUND: preprint. — arXiv, 2026. — arXiv:2602.06735.

- [8] Bachmann C., Maximova A., Kohn T., Komm D. WebTigerPython: A Low-Floor High-Ceiling Python IDE for the Browser: preprint. — arXiv, 2024. — arXiv:2410.07001.
- [9] Yan Y., Tu T., Zhao L., Zhou Y., Wang W. Understanding the Performance of WebAssembly Applications // Proceedings of the 21st ACM Internet Measurement Conference (IMC). — New York: ACM, 2021. — P. 533–549. — DOI: 10.1145/3487552.3487827.
- [10] De Macedo J., Abreu R., Pereira R., Saraiva J. WebAssembly versus JavaScript: Energy and Runtime Performance // International Conference on ICT for Sustainability (ICT4S). — IEEE, 2022. — P. 24–34. — DOI: 10.1109/ICT4S55073.2022.00014.
- [11] Jangda A., Powers B., Berger E. D., Guha A. Not So Fast: Analyzing the Performance of WebAssembly vs. Native Code // Proceedings of the 2019 USENIX Annual Technical Conference (USENIX ATC). — USENIX Association, 2019. — P. 107–120.
- [12] Wang W. Empowering Web Applications with WebAssembly: Are We There Yet? // Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering (ASE). — IEEE, 2021. — P. 1301–1305. — DOI: 10.1109/ASE51524.2021.9678831.
- [13] Van Hasselt M., Huijzendveld K., Noort N., De Ruijter S., Islam T., Malavolta I. Comparing the Energy Efficiency of WebAssembly and JavaScript in Web Applications on Android Mobile Devices // Proceedings of the 26th International Conference on Evaluation and Assessment in Software Engineering (EASE). — New York: ACM, 2022. — P. 140–149. — DOI: 10.1145/3530019.3530034.
- [14] Zakai A. Emscripten: An LLVM-to-JavaScript Compiler // Companion to the 26th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA). — New York: ACM, 2011. — P. 301–312. — DOI: 10.1145/2048147.2048224.
- [15] Herrera D., Chen H., Lavoie E., Hendren L. WebAssembly and JavaScript Challenge: Numerical Program Performance Using Modern Browser

Technologies and Devices: Technical Report SABLE-TR-2018-2. — Montreal: McGill University, 2018.

- [16] Rourke M. Learn WebAssembly. — Birmingham: Packt Publishing, 2018. — 326 p.
- [17] GeoGebra Classic [Электронный ресурс] // GeoGebra. — URL: <https://www.geogebra.org/classic> (дата обращения: 11.05.2026).
- [18] Matrixcalc.org [Электронный ресурс]. — URL: <https://matrixcalc.org/> (дата обращения: 11.05.2026).
- [19] WolframAlpha [Электронный ресурс] // Wolfram Research. — URL: <https://www.wolframalpha.com/> (дата обращения: 11.05.2026).
- [20] Octave Online [Электронный ресурс]. — URL: <https://octave-online.net/> (дата обращения: 11.05.2026).
- [21] WebAssembly Core Specification [Электронный ресурс] // W3C WebAssembly Working Group. — URL: <https://www.w3.org/TR/wasm-core-1/> (1.0, 2019); <https://www.w3.org/TR/wasm-core-2/> (2.0 Recommendation Draft, 2024–2025) (дата обращения: 11.05.2026).
- [22] HTML Living Standard [Электронный ресурс] // WHATWG. — URL: <https://html.spec.whatwg.org/> (дата обращения: 11.05.2026).
- [23] ECMAScript Language Specification [Электронный ресурс] // Ecma International. — URL: <https://tc39.es/ecma262/> (дата обращения: 11.05.2026).
- [24] Blackford L. S., Demmel J., Dongarra J., Duff I., Hammarling S., Henry G., Heroux M., Kaufman L., Lumsdaine A., Petitet A., Pozo R., Remington K., Whaley R. C. An Updated Set of Basic Linear Algebra Subprograms (BLAS) // ACM Transactions on Mathematical Software. — 2002. — Vol. 28, No. 2. — P. 135–151. — DOI: 10.1145/567806.567807.
- [25] Anderson E., Bai Z., Bischof C., Blackford S., Demmel J., Dongarra J., Du Croz J., Greenbaum A., Hammarling S., McKenney A., Sorensen D.

- LAPACK Users' Guide. — 3rd ed. — Philadelphia: Society for Industrial and Applied Mathematics (SIAM), 1999. — 407 p.
- [26] Xianyi Z., Qian W., Yunquan Z. Model-driven Level 3 BLAS Performance Optimization on Loongson 3A Processor // IEEE 18th International Conference on Parallel and Distributed Systems (ICPADS). — IEEE, 2012. — P. 684–691. — DOI: 10.1109/ICPADS.2012.97.
- [27] Kocher P., Horn J., Fogh A., Genkin D., Gruss D., Haas W., Hamburg M., Lipp M., Mangard S., Prescher T., Schwarz M., Yarom Y. Spectre Attacks: Exploiting Speculative Execution // 40th IEEE Symposium on Security and Privacy (S&P). — IEEE, 2019. — P. 1–19. — DOI: 10.1109/SP.2019.00002.
- [28] Веб-платформа интерактивной визуализации и анализа многомерных численных алгоритмов [Электронный ресурс]. — URL: <https://algoplatform.ru>.
- [29] Исходный код веб-платформы интерактивной визуализации и анализа многомерных численных алгоритмов [Электронный ресурс] // GitHub. — URL: [https://github.com/nn1125/web\\_platfrom](https://github.com/nn1125/web_platfrom).